

補助シート（指導用） — ショッピングサイト体験授業

この資料は **先生用** です。授業の流れ、配布版（穴埋め版）の作り方、各課題の模範解答と解説、トラブル対応をまとめています。生徒には配らないでください。

1. このプロジェクトの全体像

ファイルは大きく **3系統** に分かれています。

系統	URL	ファイル	役割
完成版 のお店	<code>/, /shop, /item/:code, /cart, /complete</code>	<code>views/index.ejs</code> （直下）、 <code>views/system/</code> の <code>shop.ejs, item.ejs, cart.ejs,</code> <code>complete.ejs</code>	「本物のお店」。 下の共通パーツを <code>include</code> して画 面を組み立てる。
学習用 ステッ プ	<code>/steps, /step1~/step5</code>	<code>views/system/</code> の <code>index_steps.ejs,</code> <code>step1.ejs~step5.ejs</code>	仕組みを1つずつ 学ぶ練習用ペー ジ。お店と同じ共 通パーツを <code>include</code> する。
共通パ ーツ (生徒 が穴埋 め)	—	<code>views/mikansei/product_card.ejs,</code> <code>item_detail.ejs, cart_table.ejs,</code> <code>buy_button.ejs, order_table.ejs</code>	カード／詳細／カ ート表／購入ボタ ンなど。ここを直 すとステップ画面 とお店の両方が同 時に変わる。

📁 **views/system/ フォルダについて**：生徒が編集しないページ枠（`step1~5.ejs`・`shop.ejs`・`cart.ejs` など）は `views/system/` にまとめてあり、生徒が開く `views/` 直下には `index.ejs` と `mikansei/` だけが見えるようにしています。生徒には「`system` フォルダは先生のしくみ。さわらなくてOK」と伝えてください。（ルーティングの `res.render('system/step2')` 等でこのフォルダを参照しています。）

ねらい：「自分が作った画面が、そのままお店で使われる」体験。生徒が共通パーツ（`views/mikansei/`）を穴埋めすると、練習用の Step 画面だけでなく 本番の店（`/shop`・`/item/:code`・`/cart`・`/complete`）にも即反映されます。

⚠ 裏を返すと、共通パーツを大きくこわすとお店の該当画面も一緒にくずれます。困ったら `run_reset.bat` で配布時の状態に戻すか、`git` で元に戻してください。店のレイアウト枠（`shop.ejs` など `include` する側）と Step1（`seed.js`／商品管理フォーム）は 共通パーツではないので、こわしても他画面には波及しません。最後の発展課題で、学んだことを **店名・商品**（`index.ejs` / `header.ejs` / `seed.js`）に反映させます。

★ **5ステップの流れ**：①商品登録 → ②カード表示＋詳細ページ → ③カートに入れる → ④購入する（在庫が減る） → ⑤購入完了。旧「テーブル一覧」と「在庫確認画面」は廃止／後回し。

データベースについて

- `shop.db` が商品 (`item`) とカート (`cart`) を保存しています。
- `db/seed.js` に書いた商品が「最初のデータ」です（初回起動時のみ読み込み）。
- データを入れ直したいときは `run_reset.bat` で起動（内部で `app.js --initdb`）。

2. 授業の流れ（90分・目安）

時間	内容
0-10分	<code>run.bat</code> で起動 → 完成版のお店を見る／触る（ゴールの確認）
10-20分	Step1： <code>seed.js</code> ／商品管理フォームで商品を登録 → <code>run_reset.bat</code> で反映
20-35分	Step2：カードの穴埋め（値段・在庫）と商品詳細ページ
35-50分	Step3：カートボタンの穴埋め
50-65分	Step4：小計の穴埋め＋購入して在庫が減るのを観察
65-70分	Step5：購入完了画面
70-85分	発展：トップページの店名・キャッチコピー・商品を自分の店に
85-90分	ふりかえり・友だちの店を見学

3. 配布版（穴埋め版）の作り方

完成版のリポジトリでは、`step?.ejs` は **すでに正解が書かれています**。生徒に配る前に、下表の箇所を _____（アンダーバー）に置きかえてください。（VSCode の検索で対象の1行を見つけ、該当部分だけ差しかえるのが速いです）

💡 全部を穴埋めにする必要はありません。クラスのレベルに合わせて「Step2の画像だけ」「Step4だけ」など、減らす量を調整してください。これが「データ（穴）を一個ずつ減らす／ふやす」運用です。

Step2 `views/mikansei/product_card.ejs`（共通パーツ・カード）

完成版（正解）	配布版（穴埋め）
<code>¥<%= item.price.toLocaleString() %></code>	<code>¥<%= _____ %></code>
<code>在庫 <%= item.stock %> 個</code>	<code>...在庫 <%= _____ %> 個...</code>

Step3 `views/mikansei/product_card.ejs`（Step2 と同じ共通パーツ）

カート行きリンクは **行き先・商品番号・色** の3か所が穴埋めです（href に2つ、class に1つ）。

完成版（正解）

配布版（穴埋め）

```
<a href="<%= addurl %>/<%= item.code %>"
class="btn btn-primary w-100">
```

```
<a href="<%= _____ %>/<%= _____
%>" class="btn _____ w-100">
```

3-A=addurl（行き先の箱・クォートなし）／3-B=item.code（商品番号）／3-C=btn-primary 等の色クラス。addurl は呼び出し側で /cart/add（Step3）か /shop/add（本番）に切り替わるので、生徒は変数名を書くだけ。

Step4 views/mikansei/cart_table.ejs（共通パーツ・カート表）

完成版（正解）

配布版（穴埋め）

```
<%= (cart.count * cart.price).toLocaleString() %>円 <%= _____ %>円
```

Step4 views/mikansei/buy_button.ejs（共通パーツ・購入ボタン）

「購入する」ボタンの送信先・色の2か所が穴埋めです。

完成版（正解）

配布版（穴埋め）

```
<form action="<%= buyurl %>"
method="POST">
```

```
<form action="<%= _____ %>"
method="POST">
```

```
<button type="submit" class="btn btn-
danger btn-lg px-5"
```

```
<button type="submit" class="btn
_____ btn-lg px-5"
```

4-B=buyurl（送信先の箱・クォートなし。Step4 は /purchase、本番カートは /buy に自動で切替）／4-C=btn-danger 等の色クラス。

💡 **重要な設計：** Step の画面と完成版の店は、同じ共通パーツ（views/mikansei/ 配下）を読み込んでいます。つまり生徒が上のパーツを直すと、ステップ画面と本番の店（/shop・/item/:code・/cart・/complete）が同時に変わります。「自分が作った画面がそのままお店で使われる」のがねらいです。

パーツ	使われる画面	渡している値の違い
product_card.ejs	Step2 / Step3 / /shop / トップ	showCart（ボタン有無）, addurl (/cart/add か /shop/add), showDetail （詳細リンク有無）
item_detail.ejs	Step2の「詳細」 / /item/:code	addurl（/cart/add か /shop/add）
cart_table.ejs	Step4 / /cart	removeurl（/cart/del か /cart/remove）
buy_button.ejs	Step4 / /cart	buyurl（/purchase か /buy）, total
order_table.ejs	Step5 / /complete	（差分なし）

⚠ `shop.ejs / item.ejs / cart.ejs / complete.ejs` 自体（パーツを `include` している側）は穴埋めにしないでください。穴埋めはあくまで `views/mikansei/` のパーツ側で行います。

4. 各課題の模範解答と「なぜそう書くか」

Step1 (`db/seed.js` / 商品管理フォーム)

- `seed.js` : 1行が商品1つ。VALUES ('名前', 値段, '画像名', 在庫, '説明文')。
- **なぜ再起動が必要?** `seed` は初回だけ読み込むため。`run_reset.bat` で `--initdb` が付き、`shop.db` を消して `seed` から作り直します。フォーム (`/step1`) からの追加は即時反映。
- 画像名は拡張子なし (`guitar`)。 `public/images/guitar.png` を指す。実在する名前のみ可。
- `description` (商品説明) 列は商品詳細ページに表示されます。空でもOK。

Step2 (`views/mikansei/product_card.ejs`)

```
¥<%= item.price.toLocaleString() %>  
在庫 <%= item.stock %> 個
```

- `<%= %>` は「中身を画面に出す」記号。`toLocaleString()` は数値を `30,000` のようにカンマ区切りで表示するメソッド (付けなくても動くが見やすさのため)。
- 「詳細を見る」リンク (`showDetail` が `true` のとき出る) は `/item/<%= item.code %>` を開き、共通パーツ `item_detail.ejs` で大きな画像・説明文・在庫を表示します。

Step3 (`views/mikansei/product_card.ejs`)

```
<a href="<%= addurl %>/<%= item.code %>" class="btn btn-primary w-100">
```

- **3-A `addurl`** (行き先): 呼び出し側 (Step3 なら `/cart/add`、本番ショップなら `/shop/add`) から渡される変数。`'dummy'` をクォートなしの `addurl` に書きかえる。同じカードのパーツが、画面によって行き先を変えられるしくみ。ここを文字列でハードコードすると Step3/本番のどちらかが壊れるので注意。
- **3-B `item.code`** (商品番号): リンク先は `/cart/add/3` のように番号が入る。
- **3-C 色クラス** (`btn-primary` など): `btn` の後ろに付ける色名でボタンの色が決まる。`'dummy'` のままだと `class="btn dummy w-100"` となり、色のつかない地味なボタンになる (= 未完成の合図)。
- ボタンを押すと `routes/shop.js` の `/cart/add/:code` が呼ばれ、`db.addToCart(code)` でカートに追加 → カート (Step4) へ。

Step4 (`views/mikansei/cart_table.ejs` + `buy_button.ejs` + 購入処理)

```
<%= (cart.count * cart.price).toLocaleString() %>円
```

- **4-A 小計**=数量 × 値段。() でかけ算してから `.toLocaleString()` を付ける。
- 合計は `routes/shop.js` 側の `db.calcCartTotal()` で計算済み (生徒は触らない)。
- **4-B buyurl** (`buy_button.ejs` の `form action`) : 呼び出し側から渡る送信先。Step4 は `/purchase`、本番カートは `/buy`。'dummy' をクォートなしの `buyurl` に書きかえる。未記入だと `action="dummy"` となり、押しても 404 (買えない)。
- **4-C 色クラス** (`buy_button.ejs` のボタン) : `btn-danger` など。買うボタンは目立つ色を推奨。
- **4-D 在庫を減らす** (`db/database.js` の `purchase()`) : `const newStock = item.stock - item.count;` が正解。`db.purchase()` が在庫を `stock - 数量` で更新し、カートを空にする。在庫が0だと「売り切れ」表示になり、購入はエラーメッセージで止まる。
- 「購入する」は `POST /purchase` (学習用) または `POST /buy` (完成版)。

Step5 (`views/mikansei/order_table.ejs`)

- 購入が成功すると、買った商品をセッションに保存して購入完了画面 (`/step5 / 本番 /complete`) へ。
- 明細表は共通パーツ `order_table.ejs`。在庫が減ったかは Step2 のカードの「在庫〇個」で確認。
- (旧Step6の「在庫確認テーブル」は後回しにしたため、現在は出していません。)

5. クラス運用のヒント

「データを一個ずつ減らす」

- 配布版の穴埋め数を調整して難易度を変える (§3)。
- または `seed.js` の商品数を学年・進度に応じて増減して配る。

「他の生徒と同じサイトを操作する」

- このアプリは **1つの `shop.db` をみんなで共有** する作り (在庫・カートが全員共通)。
- 同じネットワークで、生徒Aの起動したサイトに、生徒Bが `http:// (AのPCのIPアドレス) :3000` でアクセスすれば、同じ店を一緒に操作できます。
 - PCのIPは黒い画面で確認、またはコマンド `ipconfig`。
 - ファイアウォールの許可が必要な場合があります (要・事前確認)。
- ⚠ **注意** : カートはログイン別ではなく **店ごと共通**なので、複数人で同時に 同じ店のカート进行操作すると中身が混ざります。「観察用デモ」として使うのがおすすめです。各自で完結させたい場合は、生徒ごとに自分のPCでサイトを起動させてください。

6. トラブルシューティング

症状	原因・対処
画像が出ない (枠だけ)	画像名のミス。 <code>public/images/</code> に実在する名前か確認 (拡張子なしで指定)。
<code>seed.js</code> を変えたのに反映されない	<code>run.bat</code> ではなく <code>run_reset.bat</code> で起動 (初回のみ読込のため)。

症状	原因・対処
起動してすぐ黒い画面が消える	エラーの可能性。 <code>run.bat</code> (<code>pause</code> 付き) で起動しエラー文を読む。
<code>EADDRINUSE</code> (ポート使用中)	別の黒い画面でまだ起動中。古い方を閉じてから起動。
画面が真っ白/500エラー	直前に編集した <code>step?.ejs</code> の <code><% %></code> の閉じ忘れが多い。元に戻す。
カートがおかしい	<code>run_reset.bat</code> で初期化 (<code>cart</code> テーブルも作り直し)。

7. 今回の変更点 (完成版の店を追加)

📁 **【最新】** ページ枠を `views/system/` に集約：生徒が編集しないページテンプレート (`step1~5.ejs`・`shop.ejs`・`cart.ejs`・`complete.ejs`・`item.ejs`・`index_steps.ejs`・`about.ejs` など) を `views/system/` に移動しました。生徒が開く `views/` 直下は `index.ejs` と `mikansei/` だけ。これにともない `routes/shop.js` の `res.render(...)` は `'system/step2'` のように `system/` 付きに、移動した各ファイル内の `include` は `'../mikansei/...'` に変更済みです。(以下の履歴に出てくる `views/shop.ejs` 等は、現在は `views/system/shop.ejs` です。)

トップページを「Step案内メニュー」から「本物のお店」に変えました。

- `routes/shop.js`：/ (表紙) , /shop (商品一覧) , /shop/add/:code (カート追加) , /cart (カート) , /cart/remove/:code (削除) , POST /buy (購入) , /complete (完了) を追加。Step案内メニューは /steps に移動。
- `views/index.ejs`：お店の表紙 (バナー+おすすめ商品+「買い物始める」) に作り替え。
- `views/index_steps.ejs`：旧トップ (Step案内メニュー) を移設。/steps で表示。
- `views/shop.ejs` / `cart.ejs` / `complete.ejs`：完成版の店ページ (枠だけ。中身は共通パーツを `include`)。
- `views/mikansei/product_card.ejs` / `cart_table.ejs` / `order_table.ejs` / `stock_table.ejs`：Step画面と店で共有する共通パーツ (生徒の穴埋め対象)。新規追加。
- `views/step?.ejs`：重複していたHTMLを上の共通パーツの `include` に置き換え。
- `views/mikansei/header.ejs`：ナビを「Home / ショップ / カート / 商品管理 / 制作ステップ▼」に変更、店名を変更。
- `run_reset.bat`：--initdb 付き起動 (データ初期化) を追加。

7.1 ステップ構成を6→5に整理 (最新)

買い物の流れに合わせて、ステップを ①商品登録 → ②カード表示+詳細 → ③カートに入れる → ④購入 (在庫が減る) → ⑤購入完了 に再編しました。

- 旧Step2「テーブル一覧表示」を廃止 (実際の通販ではカード型しか使わないため)。
- 商品詳細ページ `/item/:code` を新規追加 (カードの「詳細を見る」から開く)。共通パーツ `views/mikansei/item_detail.ejs`、ページは `views/item.ejs`。
- `item` テーブルに `description` 列を追加 (`db/database.js` のスキーマ+既存DB向けの安全な `ALTER`、`db/seed.js` に説明文)。商品管理フォーム (`/step1`) にも「商品説明」欄を追加。
- 「在庫確認テーブル」は後回しにして、購入完了画面 (`step5/complete.ejs`) から外しました (`stock_table.ejs` は削除)。在庫の増減はカードの「在庫〇個」で確認できます。

- ステップ番号の付け替えにともない、[/cart/add](#) → [/step4](#)、[POST /purchase](#) → [/step5](#) に遷移先を変更。旧URL [/step6](#)・[/list](#)・[/cardlist](#) は互換のためリダイレクトを残しています。

関連ファイル：生徒用ワークシートは [worksheet.md](#)、生徒用の用語ガイドは [用語ガイド_生徒用.md](#)。